
INSTITUTO POLITÉCNICO DE LEIRIA
ESCOLA SUPERIOR DE TECNOLOGIA E
GESTÃO



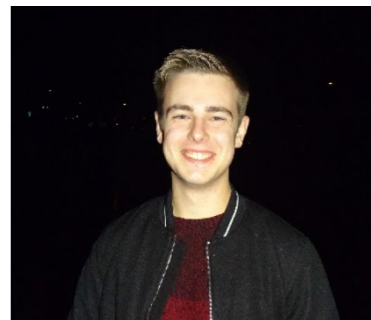
Automação Industrial 2

TRABALHO LABORATORIAL 3

Realizado por:



Judá Imbó Nº: 2192274



Maxim Postolachi Nº: 2181096

ÍNDICE

1. Introdução -----	3
2. Alterações Efetuadas -----	4
3. Especificações Funcionais -----	6
3.1 Redes de Comunicação -----	6
3.2 HMI na Consola KTP600 -----	9
3.3 HMI no Citect SCADA -----	11
3.4 Base de dados MySQL -----	13
3.5 Web Server -----	15
4. Conclusões -----	17

1.INTRODUÇÃO

Na consequência do Trabalho prático 2, onde foi completamente automatizado um sistema de embalagem e etiquetagem de caixas, considera-se ainda a implementação de várias funcionalidades. Estas funcionalidades são a comunicação entre dispositivos por PROFIBUS-DP, o HMI externo realizado no Citect SCADA, HMI na consola KTP600 e uma base de dados MySQL. A finalidade desta implementação é compreender e familiarizar-se com a utilização destas.

2. ALTERAÇÕES EFETUADAS

Voltou a haver alterações no circuito, a começar pelas funções Control_TD e Control_TE que agora faz a transferência de dados fora das condições de ativação de memórias relacionadas com a transferência de caixas. Também algumas saídas passaram a funcionar com F_TRIG, sendo agora necessário que a caixa deixe de ser detetada pelo sensor para ter um valor lógico igual a '1'.

```
#F_TRIG_Instance(CLK:="d_a",Q=>#Temp_d_a);
[]#F_TRIG_Instance_1(CLK:="x_out",
-                       Q=>#Temp_x_out );
[]#R_TRIG_Instance_2(CLK:="d_c",
-                       Q=>#Temp_d_c);

[]#F_TRIG_Instance_4(CLK:="b_out",
-                       Q=>#Temp_b_out);

IF ("Mem_Start_XA" OR "Mem_Start_XC") AND #Temp_x_out THEN

|   "Transfer_dados"(arrOrigem := "Dados".arrTapX,
-                       arrDestino := "Dados".arrTapD,
-                       iContO := "Dados".iContX,
-                       iContD := "Dados".iContD,
-                       iMaxO := "Dados".iMAXX,
-                       iMaxD := "Dados".IMAXD);

.

ELSIF ("Mem_Start_XA" OR "Mem_Start_BA") AND #Temp_d_a THEN

|   "Transfer_dados"(arrOrigem := "Dados".arrTapD,
-                       arrDestino := "Dados".arrTapA,
-                       iContO := "Dados".iContD,
-                       iContD := "Dados".iContA,
-                       iMaxO := "Dados".IMAXD,
-                       iMaxD := "Dados".iMAXA);

.
```

Houve também alteração na função de embalamento na transferência do tapete C para F, ficando igual às outras funções de transferência de caixas.



Para além destas alterações, todas as funções com variáveis do tipo Int passaram a ser Unsigned Int (UInt).

	Name	Data type	Start value
1	Static		
2	iContX	UInt	0
3	iContA	UInt	0
4	iContB	UInt	0
5	iContC	UInt	0
6	iContF	UInt	0
7	iContY	UInt	0
8	iContD	UInt	0
9	iContE	UInt	0
10	iMAXX	UInt	7
11	iMAXA	UInt	4
12	iMAXB	UInt	4
13	iMAXC	UInt	3
14	iMAXY	UInt	5
15	IMAXF	UInt	1
16	IMAXD	UInt	1
17	IMAXE	UInt	1

3. ESPECIFICAÇÕES FUNCIONAIS

3.1 REDES DE COMUNICAÇÃO

Procura-se conseguir estabelecer uma comunicação entre dois autómatos, S7-1200 e S7-1500, como também com uma estação remota, ET200L. Esta comunicação presume um dispositivo ser responsável pela transferência de dados de um dispositivo para outro, chamado “Master”. Os restantes dispositivos chamam-se “slave”. Nesta hierarquia, os “slaves” não conseguem comunicar, diretamente, entre si, necessitando primeiro enviar os dados para o Master, que depois enviará para o dispositivo entendido.

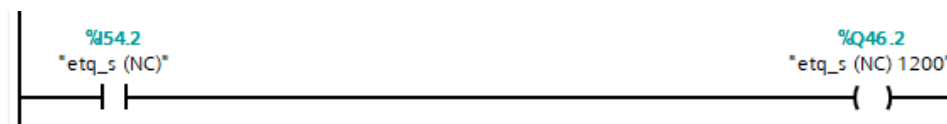
Começando pelo dispositivo principal, este é o autómato S7-1500. Os restantes são slaves.

No TIA Portal, primeiro, é preciso de estabelecer a dada ligação entre os dispositivos mencionados. Depois de adicionar ao projeto do S7-1200, o S7-1500 e o ET200L, no separador “Devices & networks” ligam-se eles os três.

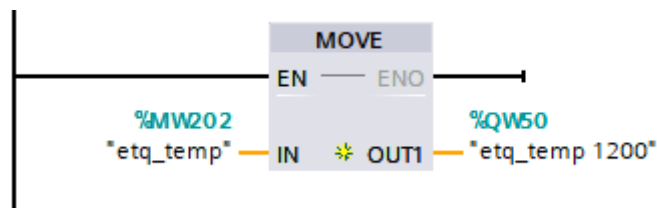
Durante a ligação dos dispositivos, foram reservados 8 bytes de memória para a comunicação entre os autómatos, começando no byte 46.0 e acabando no 53.7. Quanto á comunicação entre o ET200L com o Master, sabemos, por omissão, que as entradas do ET200L são lidas como entradas, também, no S7-1500. Por exemplo, o I0.0 no ET200L é o I54.0. O mesmo acontece para as saídas.

Todas as variáveis desta comunicação terão a sua origem ou finalidade no S7-1200.

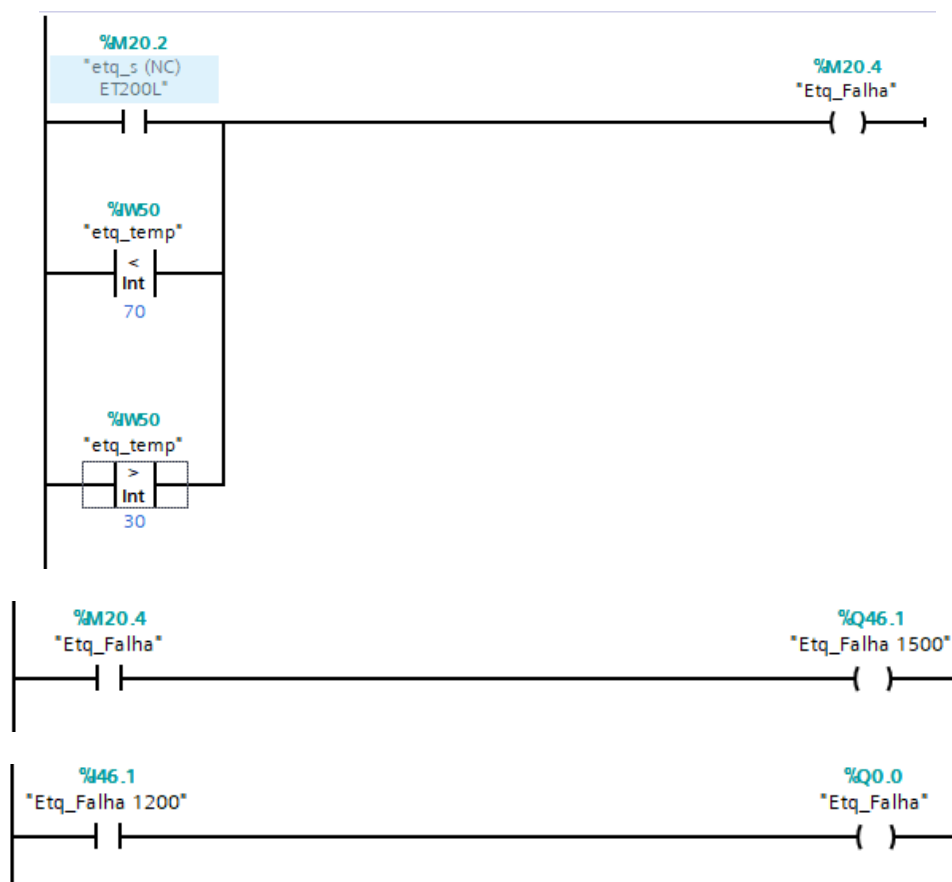
Na etiquetagem existem 2 sensores, um de falha outro de temperatura. O detetor de falha está conectado á entrada I0.2 do ET200L, o que diretamente nos diz que temos o mesmo sinal no bit I54.2 do S7-1500. Para ser enviado para o S7-1200, este bit é ligado a uma saída conectada a ele, neste caso, Q46.2.



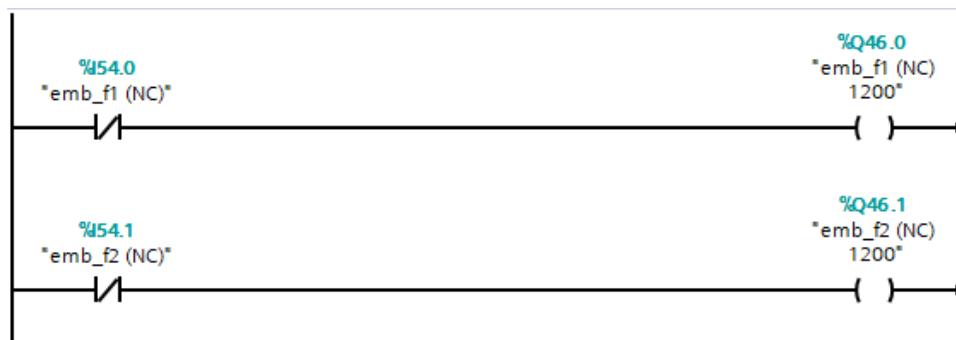
Quanto á temperatura, guardada numa memória do tamanho de 2 bytes, MW200, no S7-1500, tem que ser transferida através de um bloco chamado MOVE, que copia todos os bits desta memória para a variável escolhida. Esta variável também tem que ser do tamanho de 2 bytes.



Se alguma destas variáveis estiver com valores indesejados, uma saída, localizada no bit Q0.0 do S7-1500, tem de acender. A temperatura tem de se manter entre 30°C a 70°C. A memória M20.4 é diretamente ligada a uma saída do S7- 1200, Q46.1. Esta última é recebida no S7-1500 como I46.1, enviada seguidamente para Q0.0.

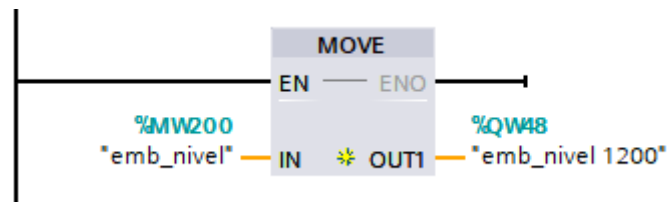


Para a verificação do funcionamento correto do sistema de embalagem temos 3 sensores e um indicador. Dois detetores de falha vêm das entradas I0.0 e I0.1 do ET200L, representados por I54.0 e I54.1, respetivamente, no S7-1500. Estes serão enviados para o S7-1200 através das saídas Q46.0 e Q46.1.

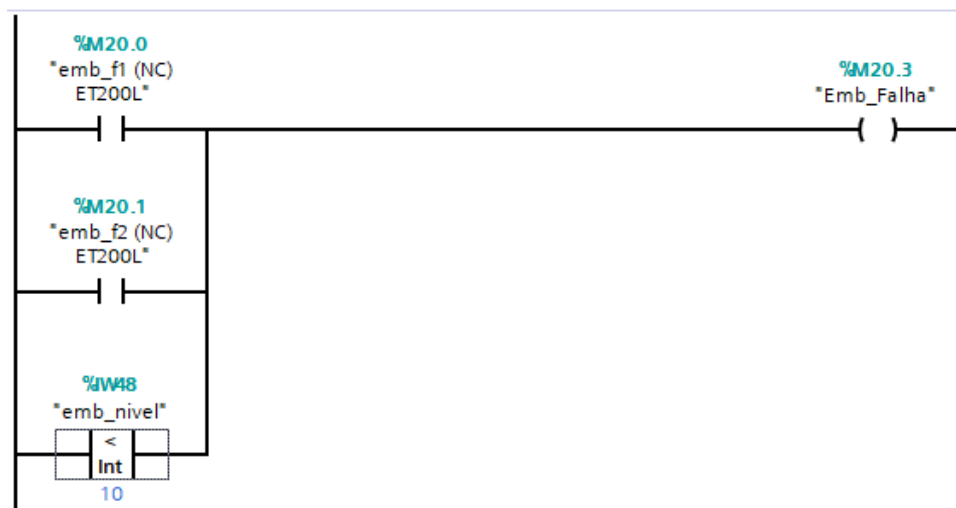


Do lado do S7-1200 estes são vistos como I46.0 e I46.1, respetivamente.

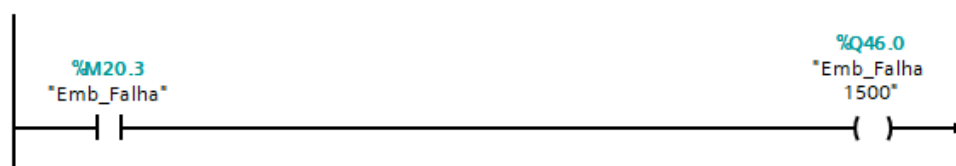
O último sensor, responsável pela disponibilização do nível do rolo, ocupa 2 bytes. Como anteriormente referido, a transferência de mais que 1 bit tem de ser feita através do bloco MOVE. Esta informação é enviada para QW48.



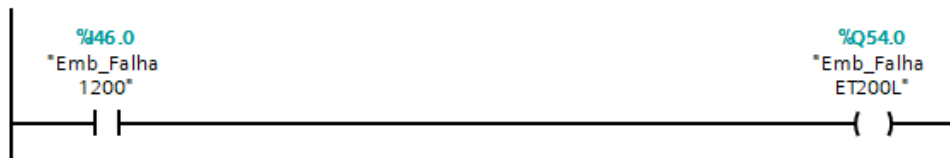
Já do lado do S7-1200, este é visto como IW48. Se algum dos sensores for ativado, ou o nível do rolo for menor que 10%, este será mostrado por uma saída no ET200L, Q0.0.



Para conseguir ligar-se ao Q0.0 do ET200L, o resultado da verificação é enviado para Q46.0 através de uma memória M20.3.

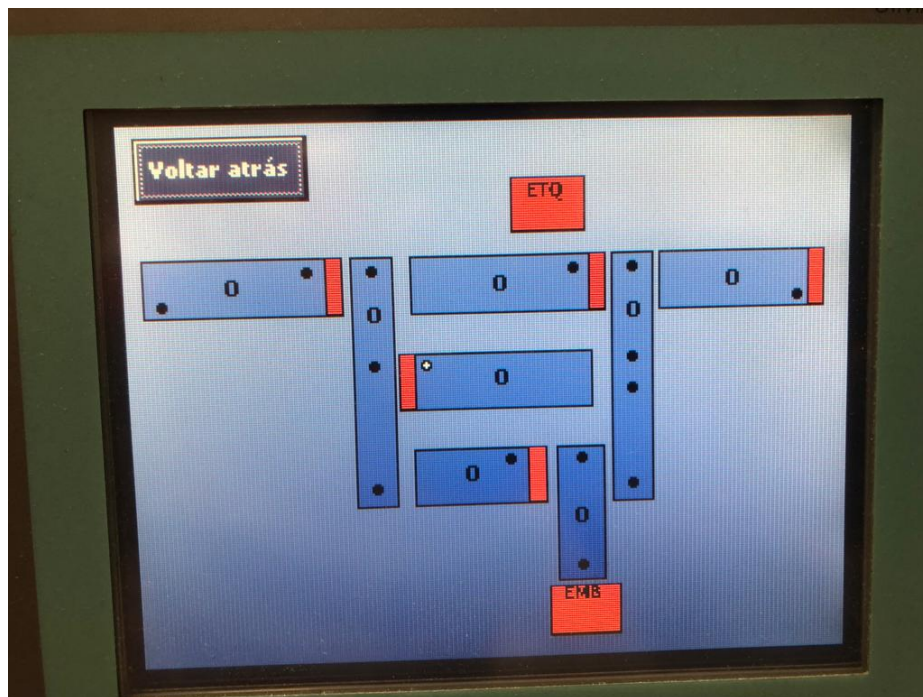


No S7-1500, este é visto como I46.0. Como o 1500 e o ET200L estão conectados diretamente, só é necessário de ligar o I46.0 ao Q54.0, que corresponde ao Q0.0 no ET200L.



3.2 HMI NA CONSOLA KTP600

O Siemens WinCC, que vem incluído no software Tia Portal, é a ferramenta utilizada para o desenvolvimento de uma tela de supervisão geral do sistema. Nele foi desenhado um esquema idêntico ao esquema dos tapetes com os sensores e atuadores, para além de também ser possível visualizar o número de caixas existentes em cada tapete.



Caso uma caixa interseje um sensor, a sua cor altera para verde, representando o estado '1' do sensor. Caso contrário, altera para preto pois a caixa deixar de ser detetada pelo

sensor. Se um atuador entrar em funcionamento, a cor deste atuador passa de vermelho para verde e vice-versa.

Para que haja estes efeitos gráficos, foi necessário referenciar as entradas e saídas que se encontram no autômato S7-1200 a outras variáveis que foram criadas no WinCC do mesmo tipo, assim quando houver uma alteração lógica no S7-1200 também haverá no HMI.

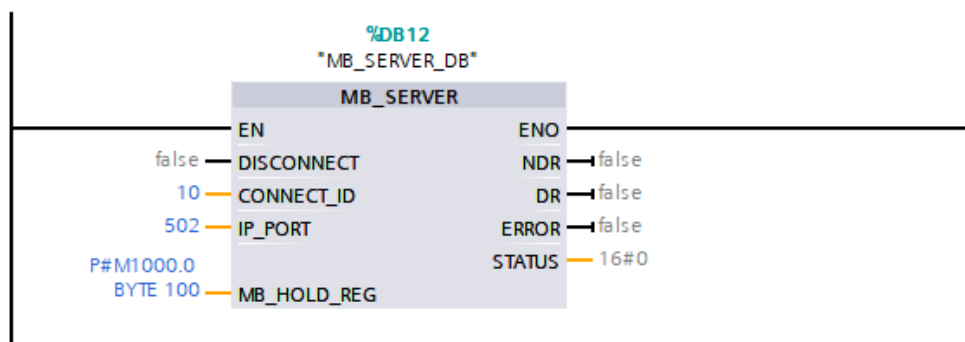
Name ▲	Data type	Connection	PLC name	PLC tag
a_out	Bool	HMI_Connectio...	PLC_1	a_out
Barreira_A	Bool	HMI_Connectio...	PLC_1	Astop
Barreira_B	Bool	HMI_Connectio...	PLC_1	Bstop
Barreira_C	Bool	HMI_Connectio...	PLC_1	Cstop
Barreira_X	Bool	HMI_Connectio...	PLC_1	Xstop
Barreira_Y	Bool	HMI_Connectio...	PLC_1	Ystop
c_out	Bool	HMI_Connectio...	PLC_1	c_out
d_a	Bool	HMI_Connectio...	PLC_1	d_a
d_b	Bool	HMI_Connectio...	PLC_1	d_b
d_c	Bool	HMI_Connectio...	PLC_1	d_c
e_a	Bool	HMI_Connectio...	PLC_1	e_a
e_b0	Bool	HMI_Connectio...	PLC_1	e_b0
e_b1	Bool	HMI_Connectio...	PLC_1	e_b1
e_c	Bool	HMI_Connectio...	PLC_1	e_c
EMB	Bool	HMI_Connectio...	PLC_1	Emb
ETQ	Bool	HMI_Connectio...	PLC_1	Etq
f_in	Bool	HMI_Connectio...	PLC_1	f_in
f_out	Bool	HMI_Connectio...	PLC_1	f_out
nCaixas_A	UInt	HMI_Connectio...	PLC_1	Dados.iContA
nCaixas_B	UInt	HMI_Connectio...	PLC_1	Dados.iContB
nCaixas_C	UInt	HMI_Connectio...	PLC_1	Dados.iContC
nCaixas_D	UInt	HMI_Connectio...	PLC_1	Dados.iContD
nCaixas_E	UInt	HMI_Connectio...	PLC_1	Dados.iContE
nCaixas_F	UInt	HMI_Connectio...	PLC_1	Dados.iContF
nCaixas_X	UInt	HMI_Connectio...	PLC_1	Dados.iContX
nCaixas_Y	UInt	HMI_Connectio...	PLC_1	Dados.iContY
x_in	Bool	HMI_Connectio...	PLC_1	x_in
x_out	Bool	HMI_Connectio...	PLC_1	x_out
y_out	Bool	HMI_Connectio...	PLC_1	y_out

Cada sensor e atuador representado por um círculo ou retângulo colorido está conectado á sua variável correspondente. Isso permite, no separador das animações, definir o que acontece quando o seu valor é '1' ou '0'.

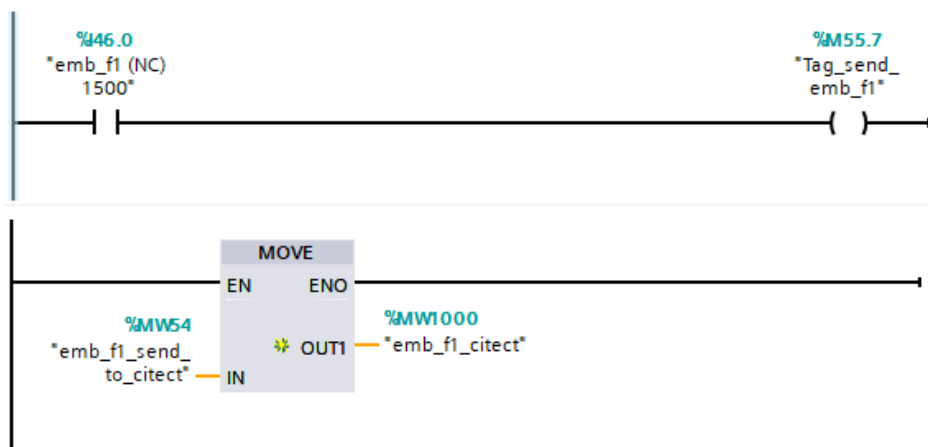
Properties	Animations	Events	Texts
Overview			
Display			
Add new animation			
Appearance			
Movements			
Appearance			
Tag			
Name: x_out			
Address:			
Type			
Range			
Multiple bits			
Single bit			
Range			
Background color			
Border color			
Flashing			
0			
1			
<Add new>			

3.3 HMI NO CITECT SCADA

O uso de um HMI externo neste trabalho requer um estabelecimento de uma conexão, entre o autômato e um computador externo, por Modbus TCP e o bloco MB_SERVER. Isso é conseguido guardando os dados, que são enviados para o Citect SCADA, nas memórias reservadas do autômato, começando no MD1000.



Para transferir os dados para a memória MD1000, criaram-se 2 memórias auxiliares: Uma para realizar a transferência de dados da memória MW64 usando o bloco MOVE ("emb_f1_send_to_citect") e outra que esteja localizada dentro da gama de memórias reservadas para o citect SCADA, para onde se deseja enviar dados, MD1000 ("emb_f1_citect").

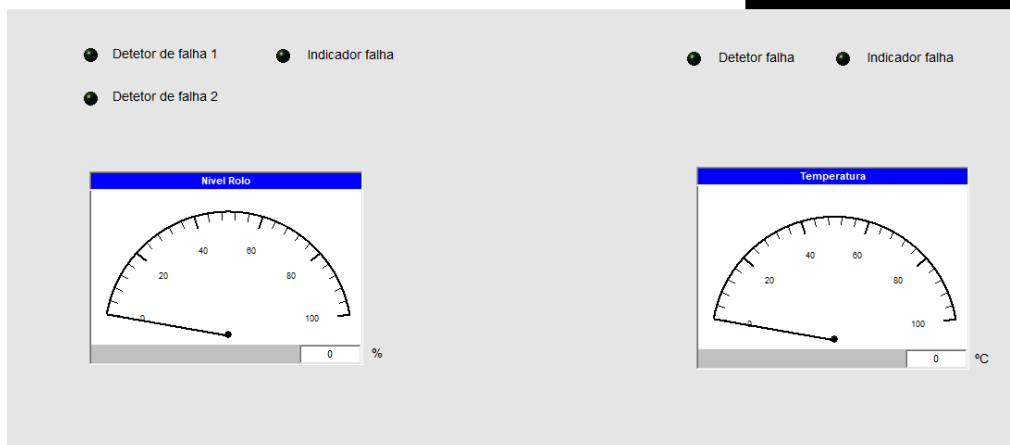


Como é enviado para o Citect variáveis do tipo BOOL ou INT, então cada variável ocupa 2 bytes, sendo que cada variável ocupa 2 bytes no autômato, tal como mostra a imagem seguinte. Já no Citect o endereçamento é normal, começando a partir do 40001, 40002 em diante.

```
Modbus 40001 – MW1000  
Modbus 40002 – MW1002  
Modbus 40003 – MW1004  
Modbus 40004 – MW1006
```

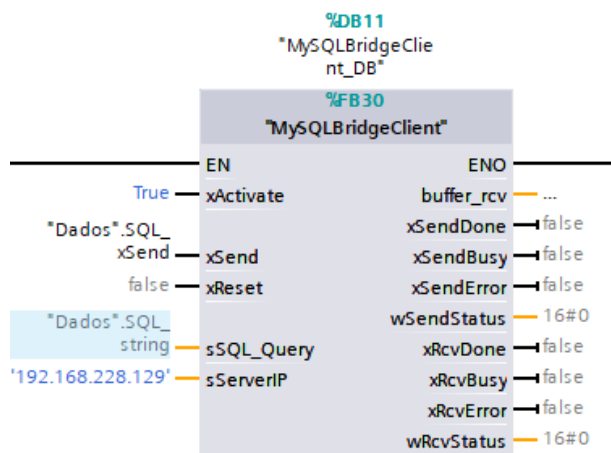
Criadas as variáveis tanto no Citect como no autômato, a interface gráfica foi criada, sendo que para alterações de estado ou mudança de temperatura, essas modificações são realizadas pelo dispositivo ET200L e pelo Tia Portal, para alterações de temperatura e nível de rolo, demonstrados no vídeo.

Trabalho Realizado por:
Maxim Postolachi nº2181096
Judá Imbó nº2192274



3.4 BASE DE DADOS MYSQL

Para que o autômato comunique com o servidor MySQL, foi necessário importar a function block MySQLBridgeClient, onde foram inseridos o código a ser enviado para o servidor (Dados.SQL_string), o sinal para enviar o código (Dados.SQL_xSend) e o endereço para onde vai ser enviado.



No software MySQL WorkBench 8.0 criou-se uma tabela com o que se quer ser registrado como o número de série, o código RFID, o tipo de caixa, os registros de etiquetagem e embalagem efetuados e a hora em que foram realizados.

	num_serie	cod_RFID	box_type	ETQ	EMB	time_etq	time_emb
➤	NULL	NULL	NULL	NULL	NULL	NULL	NULL

Existem 3 ocasiões onde é enviado um código para o servidor: Na criação de novas caixas, na embalagem e na etiquetagem, sendo que nestas 2 últimas é efetuada uma atualização no qual acrescenta novos dados, como a realização do processo e a hora em que foi efetuada.

```

"Dados".SQL_string := CONCAT(IN1 := 'INSERT INTO bd_paletes.boxes VALUES (',
                              IN2 := UINT_TO_STRING("Dados".arrTapX["Dados".iContX].N_serie),
                              IN3 := ',',
                              IN4 := UINT_TO_STRING("Dados".arrTapX["Dados".iContX].RFID),
                              IN5 := ',',
                              IN6 := INT_TO_STRING("Dados".arrTapX["Dados".iContX].Box_type),
                              IN7 := ',',
                              IN8 := 'NULL',
                              IN9 := ',',
                              IN10 := 'NULL',
                              IN11 := ',',
                              IN12 := 'NULL',
                              IN13 := ',',
                              IN14 := 'NULL',
                              IN15 := ')');

"Dados".SQL_xSend := true;

```

Function Criar_Dados

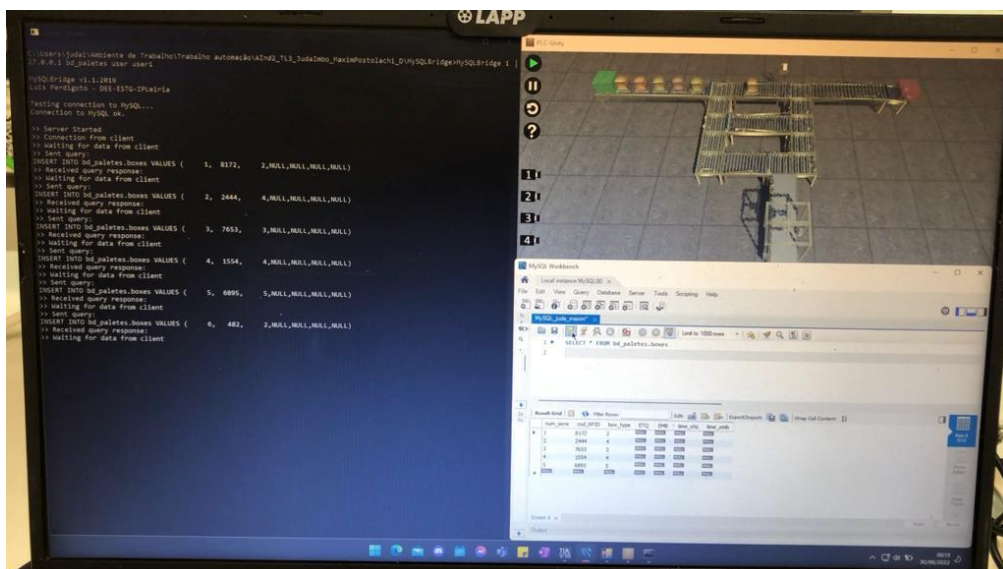
```

IF #E6 THEN
  "Dados".SQL_string := CONCAT(IN1 := 'UPDATE bd_paletes.boxes SET time_emb=CURRENT_TIMESTAMP, EMB = 1 WHERE num_serie = ',
                                IN2 := UINT_TO_STRING("Dados".arrTapF[1].N_serie));
  "Dados".SQL_xSend := TRUE;
END IF;

```

Function Block FB_Embalamento

A criação e atualização dos dados é realizado em tempo real, sendo que é possível ver todos os registos no software MySQL WorkBench 8.0.



3.5 WEB SERVER

Com uma pequena semelhança com o HMI, este permite mostrar dados adquiridos do projeto através de Tags. A diferença principal é o sítio para onde são enviados, uma página na internet, que possibilita ser vista através de um browser, não necessitando estar perto do autômato.

Primeiro têm de ser criadas Tags no autômato que guardam a informação requerida.

	W_iContX	Default tag table	UInt	%MW102
	W_iContA	Default tag table	UInt	%MW110
	W_iContB	Default tag table	UInt	%MW104
	W_iContC	Default tag table	UInt	%MW112
	W_iContD	Default tag table	UInt	%MW114
	W_iContE	Default tag table	UInt	%MW116
	W_iContF	Default tag table	UInt	%MW118
	W_iContY	Default tag table	UInt	%MW120

A seguir, utilizando as Tags definidas, cria-se o aspeto dessa página.

```
<h1>Contador de caixas:</h1>
<p>
  Tapete X = <span id="W_iContX"> </span>
</p>
<p>
  Tapete A = <span id="W_iContA"> </span>
</p>
<p>
  Tapete B = <span id="W_iContB"> </span>
</p>
<p>
  Tapete C= <span id="W_iContC"> </span>
</p>
<p>
  Tapete D= <span id="W_iContD"> </span>
</p>
<p>
  Tapete E= <span id="W_iContE"> </span>
</p>
<p>
  Tapete F= <span id="W_iContF"> </span>
</p>
<p>
  Tapete Y= <span id="W_iContY"> </span>
</p>
```

O resultado que aparece na página web do código acima pode ser vista na imagem abaixo.

Contador de caixas:

Tapete X =

Tapete A =

Tapete B =

Tapete C=

Tapete D=

Tapete E=

Tapete F=

Tapete Y=

Por fim, as Tags definidas neste editor de texto, NotePad, têm de receber um valor, o que acontece através de um comando “load”, que carrega, através de ficheiros do tipo .htm. Para cada variável é criado um ficheiro desse tipo, contendo um código pequeno que referencia a variável do autómato.

```
function autoRefresh_div(){  
<!-- Cada variável deve existir num ficheiro htm independente -->  
    $("#W_iContX").load("W_iContX.htm");  
    $("#W_iContA").load("W_iContA.htm");  
    $("#W_iContB").load("W_iContB.htm");  
    $("#W_iContC").load("W_iContC.htm");  
    $("#W_iContD").load("W_iContD.htm");  
    $("#W_iContE").load("W_iContE.htm");  
    $("#W_iContF").load("W_iContF.htm");  
    $("#W_iContY").load("W_iContY.htm");  
}  
<!-- Refrescar cada 1000ms -->  
setInterval('autoRefresh_div()', 1000);
```

1 := "W_iContA":

4. CONCLUSÕES

O trabalho foi realizado com todas as funcionalidades pedidas à exceção das funcionalidades alternativas.

A capacidade de conexão entre vários autómatos ou dispositivos é muito útil na vida profissional, algo que foi conseguido nesta última parte do projeto e o mesmo pode ser dito sobre os restantes pontos de desenvolvimento.

HMI é uma boa ferramenta para supervisionamento de qualquer processo do sistema. Existe, também, a possibilidade de controlar os processos através dos HMI, o que pode facilitar ou ajudar em várias situações.

A base de dados MySQL permitiu guardar informação sobre os processos, ativos ou não, no sistema. Quando necessário, existirá uma maneira de verificar o histórico para a procura de certos acontecimentos importantes, ou anómalos.